GNU Mach + GNU Hurd 虚拟内存架构 Tutorial

Mach VM、External Pager、Hurd 用户态内存服务与 RPC 协议

读者对象： 具备操作系统、C 语言、虚拟内存和 RPC 基础的计算机科学本科生。
学习目标： 理解 GNU Mach 暴露的虚拟内存对象和接口，理解 External Pager 的生命周期、页面读取与写回协议，以及 GNU Hurd 如何把这些能力组织成文件映射、页面缓存和交换区服务。

> **版本说明：** GNU Mach/Hurd 的接口会随版本演进。本文用于建立架构模型；实际编译或实现时，应以目标版本的 MIG 定义、头文件和源码为准。不同 Mach 派生系统的接口不可直接当作 GNU Mach 的精确 API。流程图是概念化模型，并非逐指令的内核执行轨迹。

目录

1. 整体架构
2. Mach 的基本对象
3. Mach 暴露的虚拟内存能力
4. External Pager 工作机制
5. External Pager RPC 协议
6. GNU Hurd 如何使用 Pager
7. `libpager` 的角色
8. Default Pager、Swap 与内存压力
9. 一致性、缓存、共享映射与写时复制
10. Hurd 对应用程序和其他服务暴露的接口
11. 错误处理、死锁与性能
12. 贯穿案例：一次 `mmap` 访问
13. 架构评价与设计启示
14. 源码阅读与实验路线
15. 术语表
16. 参考资料

1. 整体架构

1.1 传统宏内核的直觉

在典型宏内核中，文件系统、页缓存、虚拟内存和块设备驱动之间有大量内核内部调用：

```
应用程序
   | mmap / read / write
   v
内核 VM 与页缓存
   v
内核文件系统
   v
块设备层与驱动
   v
存储设备
```

文件映射发生缺页时，VM 子系统可以调用内核中的文件系统代码取得页面内容。

1.2 Mach + Hurd 的分层

GNU Mach 是微内核，负责 Task、线程、IPC、地址空间、页面映射和保护等低层机制。GNU Hurd 在用户态构建文件系统及其他服务。

```
+-----------------------------------------------------+
| 应用程序: mmap / munmap / mprotect / msync / read    |
+-------------------------------+---------------------+
                                |
                                v
+-----------------------------------------------------+
| Hurd 用户态服务: 文件 Translator / libpager / 存储层 |
+-------------------------------+---------------------+
                                |
                        Mach VM 与 IPC
                                |
                                v
+-----------------------------------------------------+
| GNU Mach: VM Map / 缺页 / 保护 / 页面缓存 / IPC       |
+-------------------------------+---------------------+
                                |
                                v
                          物理内存与页表
```

核心思想是：**Mach 管理地址空间和页面机制；Pager 管理内存对象的内容。** Mach 不必知道对象的文件路径、目录结构或后端存储格式。

1.3 主要角色

角色	主要责任
Client Task	映射和访问虚拟地址
GNU Mach	VM Map、缺页、页面保护、驻留页面和回收
Memory Object	可分页内容的逻辑抽象
External Pager	用户态提供或保存 Memory Object 的内容
Hurd 文件系统服务	路径、权限、文件属性、文件接口和映射入口
`libpager`	Pager 消息分派、通用协议和生命周期辅助
Default Pager	默认分页/交换支持，参与内存压力下的页面保留
存储后端	文件、交换区或其他数据来源

这些是逻辑职责，不要求每一项都对应一个独立进程。

2. Mach 的基本对象

2.1 Task 与 VM Map

Task 是 Mach 中的重要资源与地址空间抽象。Task 拥有 VM Map，并包含一个或多个线程。

```
Task
 |
 +-- VM Map
      +-- [0x1000, 0x1fff] -> Memory Object A, offset 0
      +-- [0x4000, 0x7fff] -> Memory Object B, offset 0
      +-- [0x8000, 0x8fff] -> anonymous / other object
```

VM Map 记录虚拟地址范围与底层对象的关系，并参与映射权限、继承和生命周期管理。

2.2 Memory Object

Memory Object 表示一段可由内核分页管理的逻辑内容。它不是物理内存页，也不等同于文件描述符。

内容可以来自普通文件、交换区、匿名内存或其他用户态服务管理的资源。一个对象的页面可能暂时不驻留，也可能被多个 Task 映射。

2.3 Port 与 Port Rights

Mach IPC 以 Port 为通信端点。Port Right 是一种能力：持有者可以执行该权利允许的操作。传递 Port Right 不只是传递整数 ID，而是在传递访问某个通信端点的能力。

External Pager 协议涉及不同端口角色：

端口角色	概念用途
Memory Object Port	标识 Pager 管理的对象，承载内核发给 Pager 的对象消息
Memory Control Port	初始化期间由内核提供给 Pager，用于请求内核执行相关对象管理操作
Memory Object Name Port	用于对象命名等特定用途，不应与控制端口混淆
Reply Port	在需要回复或完成通知的流程中承载回复/通知

不要假设每个对象在所有时刻必然有四个独立端口。实际关系取决于对象创建与初始化协议。

3. Mach 暴露的虚拟内存能力

3.1 地址空间和映射

常见 Mach VM 接口包括：

接口	概念作用
`vm_map`	将指定内存对象的一段内容映射到 Task 地址空间
`vm_allocate`	请求分配虚拟内存区域
`vm_deallocate`	解除指定地址范围的映射
`vm_protect`	设置映射范围的访问保护
`vm_inherit`	设置映射在任务继承中的行为
`vm_read` / `vm_write`	通过内核接口读写指定 Task 的虚拟内存，受权限与版本约束

函数参数、可用性和具体语义应以目标 GNU Mach 版本为准。Apple Mach、XNU 或其他 Mach 派生系统的接口不能直接当作 GNU Mach 的精确 API。

3.2 缺页处理

CPU/MMU 发现访问地址没有可用的驻留页面时，会产生硬件异常。内核识别该地址属于哪个映射和对象，再决定如何取得内容。如果对象由 External Pager 管理，Mach 可以向 Pager 发送页面请求。

必须区分：

- **硬件缺页**：CPU/MMU 与内核之间的异常；
- `memory_object_data_request`：Mach 与 Pager 之间的 IPC 消息。

二者有关联，但不是同一个事件。

3.3 页面保护与写时复制

Mach 管理映射保护，并在写时复制等场景中协调页面共享与私有化。

```
最初：
Task A ----+
           +---- 共享底层页面
Task B ----+

Task A 尝试写入：
Task A ---------- 私有副本
Task B ---------- 原始页面
```

具体复制策略取决于对象类型、映射方式和内核实现。

3.4 Memory Object 管理

External Pager 机制使用户态服务可以告知内核对象已准备好、响应页面内容请求、交付页面、报告数据错误或数据不可用、接收内核归还的页面、请求执行特定页面保护/锁定/缓存管理操作，并参与对象生命周期与复制策略管理。这避免内核为每一种文件系统和存储服务硬编码独立分页逻辑。

4. External Pager 的完整工作机制

4.1 为什么需要 External Pager?

External Pager 让用户态服务负责 Memory Object 的内容，Mach 负责 VM 机制：

```
应用程序
   | vm_map
   v
GNU Mach
   | memory_object_data_request
   v
External Pager
   | 读取文件 / 存储 / 其他资源
   v
数据后端
```

它不是简单的系统调用转发，而是一套围绕内存对象的 IPC 协议。

4.2 三方职责

职责	Client	Mach	Pager
普通内存访问	是	否	否
检测硬件缺页	否	是	否
管理映射和保护	通过接口请求	是	可通过协议提出相关请求
读取文件内容	否	否	是
提供页面	否	接收/安装页面	是
保存归还的脏页	否	发起归还流程	是

4.3 对象初始化

典型概念流程：

```
Client                Pager                  Mach
  |                     |                     |
  |---- 获取对象 -------->|                     |
  |                     |                     |
  |---- vm_map ------------------------------->|
  |                       |<-- memory_object_init|
  |                       |                     |
  |                       |-- memory_object_ready|
  |                       |-------------------->|
  |<--------------------- 映射继续完成 ----------|
```

步骤：

1. Client 从 Hurd 文件服务取得映射能力；
2. Client 通过 `vm_map` 请求建立映射；
3. Mach 向 Pager 发送 `memory_object_init`；
4. Pager 初始化对象状态并记录控制端口；
5. Pager 调用 `memory_object_ready`；

6. Mach 继续处理对象页面请求。

`memory_object_ready` 表示 Pager 已准备好处理对象，不代表所有页面都已载入内存。

4.4 缺页与页面提供

例如应用读取映射区域：

```
char c = ((char *)mapped)[0];
```

如果页面尚未驻留：

```
Client 访问虚拟地址
          |
          v
Mach 检测缺页
          |
          v
找到 Memory Object 与对象偏移
          |
          v
memory_object_data_request
          |
          v
Pager 读取数据
          |
          v
memory_object_data_supply
          |
          v
Mach 接收页面内容
          |
          v
恢复 Client 执行
```

Pager 根据对象偏移和长度读取后端数据。它也可能生成零页，或从其他对象取得内容。具体行为依对象类型和协议而定。

4.5 写回与回收

```
Client 修改页面
        |
        v
Mach 管理驻留页面
        |
        v
内存压力或缓存管理
        |
        v
memory_object_data_return
        |
        v
Pager 接收页面内容
        |
        v
写入文件 / 交换区 / 其他后端
```

回收过程还需要考虑页面是否被修改、写回是否完成、是否仍有映射使用页面，以及页面是否必须保留。不能将其简化为“页面不使用就立刻写盘”。

5. External Pager 的 RPC 协议

5.1 方向说明

下面按概念方向列出常见接口。某些消息是通知，不一定对应传统的请求/响应 RPC；接口可用性和细节要核对目标版本的 MIG 定义与头文件。

5.2 初始化和生命周期

接口	方向	用途
`memory_object_init`	Mach → Pager	通知对象初始化，并提供控制端口等信息
`memory_object_ready`	Pager → Mach	Pager 声明已准备好处理对象
`memory_object_terminate`	Mach → Pager	通知对象的内核管理关系结束
`memory_object_destroy`	Pager → Mach	请求销毁/终止相关内存对象管理关系

`memory_object_terminate` 和 `memory_object_destroy` 不应互换使用。

5.3 页面读取与数据提供

接口	方向	用途
`memory_object_data_request`	Mach → Pager	请求指定对象偏移和长度的页面内容
`memory_object_data_supply`	Pager → Mach	将页面数据交给内核
`memory_object_data_provided`	Pager → Mach	较旧形式的数据提供接口，需核对版本
`memory_object_data_unavailable`	Pager → Mach	按协议表示该范围没有可直接提供的数据
`memory_object_data_error`	Pager → Mach	报告指定范围的数据无法提供
`memory_object_supply_completed`	Mach → Pager	通知先前的数据提供操作完成

数据提供必须遵守页面对齐、长度、权限和对象范围等约束，不是任意字节区间的普通文件写入接口。

`memory_object_data_unavailable` 不等同于磁盘 I/O 失败。具体行为可能涉及零填充或对象复制规则，必须根据对象类型和协议上下文判断。

5.4 页面写回

接口	方向	用途
`memory_object_data_return`	Mach → Pager	将需要归还的页面内容交给 Pager
`memory_object_data_write`	Mach → Pager	旧式数据写回接口；在文档中属于过时接口

`memory_object_data_return` 可能携带页面偏移、长度、脏页状态和内核是否保留副本等信息。Pager 必须按协议处理数据和相关缓冲区生命周期。

5.5 页面锁定与属性

接口	方向	用途
`memory_object_lock_request`	Pager → Mach	请求对页面范围执行锁定、保护或相关缓存管理操作
`memory_object_data_unlock`	Mach → Pager	与页面解锁相关的请求
`memory_object_lock_completed`	Mach → Pager	通知锁定请求完成
`memory_object_set_attributes`	Pager → Mach	设置对象属性
`memory_object_get_attributes`	Pager → Mach	获取对象属性

这些协议影响页面保护、缓存和写回/回收行为，并不是普通 `mprotect()` 的简单替代品。

5.6 对象复制

`memory_object_copy` 等接口与内存对象复制及写时复制有关。应结合复制策略、新对象初始化、内核当前持有的页面和对象生命周期理解；不能简单视为复制一个普通文件。

查阅精确接口的首选来源： GNU Mach Reference Manual 的 External Memory Management / Memory Objects and Data 章节，以及目标源码版本的 MIG 定义和头文件。

6. GNU Hurd 如何使用 External Pager

6.1 从内核机制到用户态服务

Mach 只提供内存对象与分页协议，不知道对象究竟是普通文件、交换区、设备数据还是其他资源。

Hurd 在用户态构建文件系统 Translator、`libpager`、存储服务和默认分页服务等组件。它们通过 RPC 组合系统功能，而不是把所有文件系统逻辑放进微内核。

6.2 文件 Port 与 Memory Object Port

Hurd 文件服务通常通过 Port 表示文件接口。文件 Port 和 Memory Object Port 是不同层次的能力：

- **文件 Port**：用于文件系统接口，例如属性、读写和映射入口；
- **Memory Object Port**：用于参与 Mach External Pager 协议，提供文件映射所需的页面内容。

二者可以关联，但不能假设它们天然是同一个 Port。

6.3 `io_map` 的作用

Hurd 的 `io_map` 是文件服务向客户端提供映射能力的接口。GNU Hurd 文档指出，`io_map` 返回的 Port 必须实现 External Pager 内核接口，才能作为 `vm_map` 的对象参数。

```
应用程序
   |
   | 文件接口 RPC
   v
Hurd 文件系统 Translator
   |
   | io_map
   v
Memory Object Port
   |
   | vm_map
   v
```

```
文本（续）
GNU Mach
   |
   | 缺页时向 Pager 请求页面
   v
文件系统 Pager
   |
   v
文件数据后端
```

接口层次必须分清：

- `io_map`：Hurd 文件服务面向客户端提供的映射接口；
- `memory_object_*`：Mach 面向 Pager 提供的底层分页协议。

它们不是同一组 RPC。

6.4 为什么需要这一层抽象？

文件服务可以继续通过自己的 RPC 提供路径查找、权限检查、文件大小等功能，而内存映射的数据通路由 Pager 协议完成。

这使不同文件系统能够在保留自身语义的同时，复用 Mach 的 VM 机制。

6.5 `MAP_SHARED` 与 `MAP_PRIVATE`

两种映射语义有明显区别：

- `MAP_SHARED`：修改应按照共享映射语义对其他映射可见，并在适当时机传播到文件后端。
- `MAP_PRIVATE`：修改应形成私有的写时复制状态，不应直接修改原文件。

Mach 的内存对象复制策略参与实现这类语义。不能简单认为 `MAP_PRIVATE` 一定对应一个新建的独立文件或一个独立 Pager；真正实现依赖 Mach 的复制策略、对象类型和 Hurd 的映射实现。

6.6 文件大小与映射边界

文件服务需要处理三个不同边界：

1. **虚拟地址空间边界**：由 `vm_map` 建立的映射范围决定；
2. **内存对象边界**：由 Pager 能接受的有效偏移和长度决定；
3. **文件逻辑大小边界**：由文件系统维护的文件长度决定。

这三个边界不一定相同。例如文件大小为 5000 字节、页面大小为 4096 字节时，最后一个文件页可能包含文件末尾之后的填充区域。文件服务必须按文件映射语义处理这些字节，不能把页面对齐后的长度等同于文件逻辑大小。

7. `libpager`：把协议变成可实现的服务

7.1 为什么需要 `libpager`？

直接实现 External Pager，意味着服务需要处理对象初始化、页面请求、页面写回、消息分派和生命周期管理等细节。

Hurd 提供 `libpager` 帮助实现多线程 Pager。官方文档将其描述为用户态 Pager 的通用库，提供消息处理及与后端相关的回调机制。它依赖线程和端口管理设施。

参考：GNU Hurd Reference Manual — Pager Library。

7.2 创建 Pager 的概念流程

常见高层 API 包括：

```
struct pager *pager_create (...);
void pager_sync (struct pager *pager, int wait);
void pager_sync_some (...);
void pager_flush (struct pager *pager, int wait);
void pager_flush_some (...);
```

以上展示 API 名称和大致用途，不是完整可直接编译的示例。实际参数和类型应以目标版本的 `<hurd/pager.h>` 为准。

- `pager_create`：创建 Pager 管理对象；
- `pager_demuxer`：把收到的 IPC 消息分派给相应处理逻辑；
- `pager_sync`：同步 Pager 中需要持久化的数据；
- `pager_flush`：请求刷新相关页面并处理延迟复制等情况。

7.3 Pager 回调

`libpager` 把与具体存储后端有关的工作留给应用。常见回调包括：

回调	职责
`pager_read_page`	读取指定页面内容
`pager_write_page`	将指定页面写回后端
`pager_unlock_page`	处理页面可写状态相关操作
`pager_report_extent`	报告 Pager 对象的有效范围
`pager_clear_user_data`	清理用户态对象关联数据
`pager_dropweak`	处理相关弱引用生命周期

具体签名、返回值和错误码应以对应版本的 Hurd Reference Manual 与头文件为准。

以 `pager_read_page` 为例，服务根据页面偏移读取后端数据，并将有效缓冲区交给 Pager 库。随后由库完成与 Mach 的数据交付。

```
逻辑页面偏移
      |
      v
存储位置映射
      |
      v
读取数据
      |
      v
返回页面缓冲区
      |
      v
libpager 完成协议处理
```

这样，服务开发者不必在文件读取代码中重新实现整个 Mach IPC 状态机。

7.4 Pager 对象生命周期

Pager 不只是一个后台线程，它还涉及：

- Pager 自身的用户态对象；
- 关联的 Mach Port；
- 文件或其他后端对象；
- 尚未完成的页面请求；
- 正在使用对象的 Client；
- 与内核关联的 Memory Object 状态。

关闭文件句柄不意味着关联映射一定可以立即销毁。仍存在的映射可能继续使用该对象。

同样，`pager_shutdown` 也不是简单地强制结束线程。如果仍有未完成的分页请求，资源释放可能异步完成。**文件句柄生命周期、映射生命周期、Pager 生命周期和后端存储对象生命周期必须分别建模。**

8. Default Pager、Swap 与内存压力

8.1 为什么需要 Default Pager?

假设一个文件系统 Pager 负责某个文件的映射。Mach 决定回收脏页时，会通知 Pager 保存内容。但用户态 Pager 可能出现磁盘阻塞、线程耗尽、死锁或服务异常等情况。

如果内核完全依赖外部 Pager 及时响应，物理页面回收可能被用户态服务阻塞。

GNU Mach 的架构文档描述了 Default Pager 作为默认分页保障的角色。它参与内存压力下的页面保留路径。
<Cite refs={["turn984390search3","turn984390search0"]}/>

8.2 Double Paging

GNU Mach 文档描述了一种称为 **double paging（双重分页）** 的路径：当外部 Pager 不能及时释放页面时，内核可以通过 Default Pager 保留页面内容，避免外部服务无限期阻塞物理页面回收。

概念图：

```
文件 Pager
    |
    | 暂时无法释放页面
    v
Mach VM
    |
    | 默认分页保障路径
    v
Default Pager
    |
    v
Swap Store
```

这不等于把文件永久改成交换区对象。之后重新访问页面时，系统还必须恢复内容，并协调原始对象和默认分页对象之间的关系。

双重分页的具体状态机和实现细节应结合目标 GNU Mach 源码阅读，不应仅凭这张概念图推断全部执行顺序。

8.3 Swap Store 与 `storeio`

Hurd 的交换区管理涉及用户态存储服务。在官方架构图中，Default Pager 可以通过存储接口将页面内容交给 `storeio`，最终写入交换设备或存储后端。<Cite refs={["turn984390search0"]}/>

```
Mach VM
   |
   v
Default Pager
   |
   | 存储请求
   v
storeio
   |
   v
交换设备 / 存储后端
```

`storeio` 是用户态存储服务架构的一部分，不是 Mach 内核中的交换区驱动。实际交换设备配置和默认启动方式需要按具体 Hurd 版本核实。

8.4 为什么 Default Pager 不能取代所有 External Pager?

Default Pager 提供默认分页保障，但不理解所有对象语义：

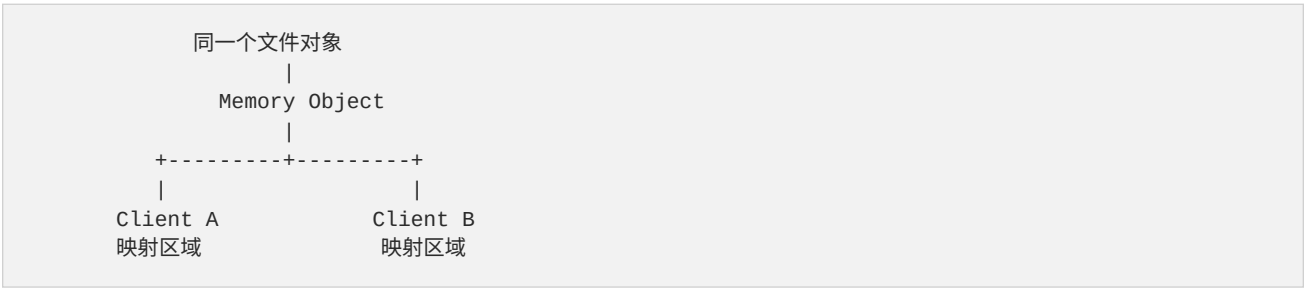
- 文件 Pager 知道文件大小、偏移和文件后端；
- 交换区 Pager 负责交换数据；
- 匿名对象有自己的内容管理语义；
- 其他用户态服务可能实现不同的对象策略。

因此，Default Pager 并不能替代这些服务的全部职责。它解决的是物理页面回收和内容保留路径中的特定问题。

9. 一致性、缓存、共享映射与写时复制

9.1 为什么需要一致性协议?

假设两个 Client 映射同一个文件：



如果 Client A 修改某个页面，Client B 应按照共享映射语义观察到相应修改。这要求内核和 Pager 正确处理：

- 多个映射之间的页面共享；
- 脏页状态；
- 页面保护；
- 写回；
- 页面缓存；
- 映射撤销和失效。

Mach 的 Memory Object 模型允许内核缓存对象页面，而 Pager 负责对象内容的管理和必要的一致性协调。

9.2 `memory_object_lock_request`

Pager 可以通过 `memory_object_lock_request` 请求内核对特定范围执行锁定、保护或相关操作。例如，Pager 可能需要限制页面写入，以便完成同步或保证操作顺序。

```
Pager
  |
  | memory_object_lock_request
  v
Mach
  |
  +-- 调整页面访问保护
  +-- 根据请求处理脏页
  +-- 处理页面缓存
  |
  v
完成通知（如适用）
```

这不表示 Pager 可以无条件绕过内核权限检查。内核仍负责管理映射保护和页面状态。

9.3 `memory_object_data_return` 不是普通 `write()`

普通文件写入：

```
write(fd, buffer, length);
```

表达调用者请求修改文件内容。

`memory_object_data_return` 则表示 Mach 将内存对象中需要归还的页面内容交给 Pager。它可能携带页面偏移、长度、脏页状态以及内核是否保留副本等信息。

两者的区别：

- `write()` 属于文件接口层；
- `memory_object_data_return` 属于分页协议层。

两者可以有关联，但不能视为同一个操作。

9.4 `MAP_SHARED` 与 `MAP_PRIVATE`

- `MAP_SHARED`：修改应按照共享映射语义对其他映射可见，并在适当时机传播到文件后端。
- `MAP_PRIVATE`：修改应形成私有的写时复制状态，不应直接修改原文件。

Mach 的内存对象复制策略参与实现这类语义。不能简单认为 `MAP_PRIVATE` 一定对应一个新文件或一个独立 Pager；实际实现依赖复制策略、对象类型和 Hurd 的映射路径。

9.5 文件大小与映射边界

必须区分三个边界：

1. **虚拟地址空间边界**：由 `vm_map` 建立的映射范围决定；
2. **内存对象边界**：由 Pager 能接受的有效偏移和长度决定；
3. **文件逻辑大小边界**：由文件系统维护的文件长度决定。

例如文件大小是 5000 字节、页面大小是 4096 字节，最后一个文件页可能包含文件末尾之后的填充区域。文件服务必须按照文件映射语义处理这些字节，不能把页面对齐后的长度等同于文件逻辑大小。

10. Hurd 对应用程序和其他服务暴露的接口

10.1 四层接口模型

```
第 1 层：应用程序接口
  mmap() / munmap() / mprotect() / msync()
                |
                v
第 2 层：Hurd 文件接口
  io_map / 文件打开、属性、读写等 RPC
                |
                v
第 3 层：Mach VM 接口
  vm_map / vm_deallocate / vm_protect 等
                |
                v
第 4 层：External Pager 协议
  memory_object_init / data_request / data_supply / data_return
```

这四层不是一一对应关系。例如 `mmap()` 由运行库和 Hurd 机制实现，可能经过文件接口和 Mach VM 调用；Pager 与 Mach 之间的消息则由缺页、写回和对象管理事件触发，不需要应用直接发起。

10.2 应用程序通常不直接调用 Pager RPC

普通应用程序通常写：

```c
void *p = mmap(NULL, length,
               PROT_READ | PROT_WRITE,
               MAP_SHARED, fd, 0);
```

应用程序不需要自己构造 `memory_object_data_request` 或 `memory_object_data_supply`。这些属于 Pager 与 Mach 之间的底层协议。

10.3 文件系统服务如何提供映射能力？

实现支持内存映射的 Hurd 文件系统时，需要考虑：

1. 文件服务如何识别并授权文件对象；
2. `io_map` 如何返回合适的 Memory Object Port；
3. Pager 如何建立对象生命周期；
4. `pager_read_page` 如何读取后端数据；
5. `pager_write_page` 如何写回修改；
6. 文件大小、截断、删除和映射并存时如何保持正确性。

`io_map` 不是让应用直接控制物理页的接口。它提供的是一个能够参与 Mach 分页协议的对象能力。

10.4 为什么其他服务也可以使用这套机制？

External Pager 并不局限于文件系统。只要用户态服务能够实现相应内存对象协议，就可以为某种资源提供分页内容。资源可以是普通文件、交换数据，或其他适合表示为 Memory Object 的内容。

不是所有 Hurd 服务都自动拥有分页能力。服务必须实现相应接口，正确管理 Port Rights 和对象生命周期，并处理同步及错误情况。

11. 错误处理、死锁与性能

11.1 Pager 读不到页面时怎么办?

如果 Mach 请求页面，而 Pager 发现后端读取失败，可以按协议使用 `memory_object_data_error` 报告相应范围的数据无法提供。对于确实没有数据且符合协议语义的情况，可以考虑 `memory_object_data_unavailable`。二者不能互换。

情况	处理方向
页面内容正常存在	读取并提供页面
后端 I/O 错误	报告数据错误
对象范围内没有数据且协议允许	按对应语义报告数据不可用
Pager 已销毁或端口失效	按对象生命周期和 IPC 错误路径处理

`memory_object_data_unavailable` 不等同于磁盘错误；某些情况下它可能允许内核按对象语义处理零填充或复制规则。

11.2 Pager 死锁为什么危险?

例如:

1. Pager 正在处理缺页请求;
2. 它需要从某个服务读取文件;
3. 该服务自身又需要访问由同一个 Pager 管理的对象;
4. 新访问再次产生缺页;
5. 两个操作互相等待。

```
Pager
  |
  v
存储服务
  |
  v
访问另一个对象
  |
  v
再次请求 Pager
  |
  +----------> 等待原操作完成
```

设计 Pager 时应考虑:

- 避免分页关键路径中不受控的递归访问;
- 避免持锁等待可能再次依赖同一把锁的操作;
- 保证处理请求的线程池不会被完全耗尽;
- 对后端 I/O 超时和失败设计明确策略;
- 对象销毁时正确处理尚未完成的请求。

11.3 Pager 崩溃意味着什么?

如果 Pager 退出、相关 Port 失效或后端无法提供数据，Mach 不可能凭空恢复所有原始内容。能否恢复取决于是否存在其他有效副本、缓存页面或后端数据。

用户态服务可替换，不等于服务状态和数据天然具备容错能力。

11.4 性能成本

External Pager 可能带来：

- 缺页时的 IPC；
- 后端读取延迟；
- 跨地址空间传递页面数据的成本；
- 大量小页面请求的消息处理开销；
- 写回、锁定和对象生命周期协调开销。

但常驻页面的普通读写通常不需要每次都经过 Pager RPC。Pager 主要参与缺页、页面提供、回收、同步和相关管理事件。实际成本取决于缺页率、缓存命中率、批量传输、后端布局和并发处理能力。

12. 贯穿案例：一次文件 `mmap` 访问

假设应用程序执行：

```
int fd = open("/data/example.bin", O_RDWR);

char *p = mmap(NULL, 8192,
               PROT_READ | PROT_WRITE,
               MAP_SHARED, fd, 0);

char c = p[100];
p[100] = 'X';
msync(p, 8192, MS_SYNC);
```

12.1 建立映射

1. 应用调用 `open()`，Hurd 文件服务返回文件接口能力。
2. 应用调用 `mmap()`。
3. Hurd 文件服务通过映射接口（如 `io_map`）提供适当的 Memory Object Port。
4. 运行库/系统接口请求 Mach 建立 VM Map 映射。
5. Mach 与 Pager 完成对象初始化握手。

此时不要求两个页面都已经驻留。

12.2 第一次读取

1. 应用读取 `p[100]`。
2. 如果对应页面不驻留，CPU 触发缺页。
3. Mach 找到对应 VM Map 区域、Memory Object 和对象偏移。
4. Mach 通过 `memory_object_data_request` 请求页面。
5. Pager 从文件后端读取对应页面。

6. Pager 通过数据提供协议交付页面。
7. Mach 安装/关联页面并恢复应用执行。

12.3 修改页面

1. 应用写入 `p[100]`。
2. 如果页面尚未具备写权限，可能触发保护异常或写时复制处理，具体取决于映射状态。
3. 对 `MAP_SHARED`，修改最终应按照共享映射语义传播。
4. Mach 跟踪相关页面状态，并在同步或回收路径上与 Pager 协作。

12.4 同步与写回

`msync()` 是应用层同步请求。它不等同于直接调用 `memory_object_data_return`。Hurd 的实现需要把应用层同步语义与文件接口、Pager 同步/刷新及底层存储写回协调起来。

如果 Pager 使用 `libpager`，可利用相应的同步和刷新接口，但具体调用链应通过所研究版本的 Hurd 源码验证。

---

13. 设计分析：微内核架构的收益与代价

13.1 收益

职责分离。 Mach 管理地址空间、页面保护和 IPC；用户态 Pager 管理对象内容和后端存储。

可扩展性。 不同对象可以使用不同 Pager，而不要求每个后端都进入内核。

故障隔离的潜力。 用户态服务比内核组件更容易独立替换或重启，但前提是协议、状态恢复和数据持久性设计正确。

统一的内存对象抽象。 文件、交换区和其他资源可以通过同一类分页协议与内核协作。

13.2 代价

IPC 与上下文切换。 缺页和写回可能需要跨地址空间通信。

生命周期复杂。 Task、VM Map、Memory Object、Port、Pager 和后端对象的生命周期相互关联。

死锁风险。 Pager 本身可能依赖其他用户态服务；这些服务若反过来访问 Pager 管理的对象，就可能形成依赖环。

一致性成本。 多个映射共享页面时，需要处理脏页、写回、保护和失效。

调试难度。 一次页面访问可能跨越应用、运行库、Mach、Pager、文件系统和存储后端多个组件。

13.3 设计原则

1. 把对象身份与对象数据来源分开。
2. 明确每个 Port Right 的拥有者和传递边界。
3. 分别建模文件句柄、映射、Memory Object、Pager 和后端数据的生命周期。
4. 将普通内存访问路径与缺页/写回慢路径区分开。
5. 设计清晰的错误传播与超时策略。
6. 不要把 `io_map` 与 `memory_object_*` 混为同一层接口。
7. 对具体版本的协议细节，始终回到 MIG 定义和实现代码验证。

---

14. 源码阅读与实验路线

14.1 建议的阅读顺序

1. 阅读 GNU Mach Reference Manual 中的 VM 与 Memory Objects 章节。
2. 查找 GNU Mach 源码中的 `memory_object_*` MIG 定义和消息分派实现。
3. 阅读 Hurd Reference Manual 中的 Pager Library。
4. 查看 `<hurd/pager.h>`，确认 `pager_create`、同步、刷新和回调的准确签名。
5. 阅读一个 Hurd 文件系统如何实现 `io_map`。
6. 跟踪文件系统 Pager 的 `pager_read_page` 与 `pager_write_page`。
7. 查看 Default Pager 与 `storeio` 的架构及实现。
8. 最后追踪 `mmap()` 从用户态接口进入 Hurd 文件接口与 Mach VM 的路径。

14.2 建议做的实验

实验 A：跟踪文件映射

创建一个小文件，用程序 `mmap()` 读取和写入，观察文件系统服务和 Pager 的消息路径。目标是理解首次访问与后续访问的差别。

实验 B：比较共享映射和私有映射

分别使用 `MAP_SHARED` 和 `MAP_PRIVATE`，修改映射内存，再检查原文件内容。关注写时复制和写回语义。

实验 C：观察页面同步

使用 `msync()` 与文件同步相关接口，阅读 Hurd 源码确认同步请求如何传递到 Pager。

实验 D：研究 Pager 生命周期

跟踪对象初始化、映射建立、映射解除以及最后一个引用消失时的清理路径。重点观察 Port Rights 和异步清理。

实验 E：阅读 Default Pager

沿着 Default Pager 到 `storeio` 的路径阅读实现，理解交换区后端与外部文件 Pager 的职责差别。

14.3 阅读源码时的检查清单

- `io_map` 的 MIG 定义是什么?
- 文件服务返回的端口具有什么类型与权限?
- 哪个代码创建 Memory Object?
- `memory_object_init` 在哪里被分派?
- Pager 的 `memory_object_ready` 何时调用?
- `memory_object_data_request` 如何转换成后端读取?
- 页面缓冲区的所有权如何转移?
- `memory_object_data_return` 处理后何时释放资源?
- 同步和刷新 API 如何传播到后端?
- Pager 退出时如何处理尚未完成的请求?

15. 术语表

术语	含义
Mach	微内核，提供任务、线程、IPC 和 VM 等机制
GNU Hurd	基于 GNU Mach 的用户态操作系统服务体系
Task	Mach 的任务/地址空间管理抽象
VM Map	Task 的虚拟地址映射结构
Memory Object	可由 Mach 分页管理的逻辑内容对象
External Pager	用户态内存对象内容管理服务
Default Pager	默认分页/交换保障机制
Port	Mach IPC 通信端点
Port Right	对 Port 的一种能力/访问权利
`vm_map`	Mach VM 映射接口
`io_map`	Hurd 文件接口中用于提供映射能力的 RPC
`libpager`	Hurd 用户态 Pager 实现辅助库
`MAP_SHARED`	共享文件映射语义
`MAP_PRIVATE`	私有映射，通常涉及写时复制
COW	Copy-on-Write，写时复制
Swap	将内存内容保存在交换区，以便释放物理页面
MIG	Mach Interface Generator，用于定义/生成 Mach IPC 接口代码

---

16. 参考资料

以下官方资料是进一步学习的首选。具体页面组织和接口细节可能随文档版本变化。

1. GNU Mach Reference Manual

GNU Mach 的 VM、Memory Object 和 External Pager 参考资料。

2. GNU Mach — External Pager Mechanism

External Pager 的架构说明。

3. GNU Mach — Memory Objects and Data

Memory Object 与数据交付相关协议。

4. GNU Hurd Reference Manual

Hurd 用户态服务、文件接口和相关机制。

5. GNU Hurd Reference Manual — Pager Library

`libpager` 的高层接口及回调说明。

6. GNU Hurd 官方网站

项目介绍、文档和开发资料。

如何使用这些资料

建议先用本文建立整体对象模型，再在官方手册中核对每个接口的参数、方向和生命周期约束。若要实现真实 Pager，必须以目标 GNU Mach/Hurd 版本的 MIG 文件、头文件和源码为准；不要仅凭概念图实现协议。

---

17. 总结

GNU Mach + GNU Hurd 的虚拟内存设计可以概括为：

1. **Mach 管理 VM 机制**：地址空间、映射、保护、缺页、驻留页面和页面回收。
2. **Memory Object 抽象内容**：把“某个地址对应什么内容”与“内容存在哪里”分开。
3. **External Pager 提供页面内容**：通过 `memory_object_*` IPC 响应页面请求、归还页面并参与对象管理。
4. **Hurd 组织用户态服务**：文件系统通过 `io_map` 等接口向客户端提供映射能力；`libpager` 帮助实现 Pager；Default Pager 与交换区处理默认分页保障。
5. **应用程序使用较高层接口**：一般使用 `mmap()`、`msync()` 等，而不直接调用底层 Pager RPC。
6. **微内核的代价来自跨边界协作**：IPC、生命周期、一致性、死锁和错误传播都需要仔细设计。

最重要的学习结论是：**Mach 并不是把所有内存管理都交给用户态，而是把地址空间与物理页面管理留在内核，同时允许用户态服务定义内存对象的内容和后端策略。** 这正是 External Pager 架构的核心。